

PLAN DE DESARROLLO PASO A PASO

OpenFang Studio

Cómo llevar la interfaz visual de OpenFang desde el esqueleto actual hasta el sistema completo, en fases verificables.

Versión del documento	1.0
Fecha	4 de octubre de 2026
Organización	Aicore Agency
Documento hermano	Especificación de Requisitos (openfang-studio-ERS.pdf)
Punto de partida	Fase 1 implementada y probada contra OpenFang v0.6.9

Contenido

- 1. Cómo usar este plan 3
 - 1.1 Estado de partida 3
 - 1.2 Mapa de fases 3
- 2. Fase 0: poner en marcha en tu VPS (Dokploy) 3
- 3. Fase 1: cerrar Mission Control 5
- 4. Fase 2: eventos, trazas y costos 5
- 5. Fase 3: constructor visual y asistente 7
- 6. Fase 4: operación (Hands, programación, canales, alertas, móvil) 7
- 7. Fase 5: confianza y escala 9
- 8. Prácticas transversales 10
 - 8.1 Actualizar el motor sin sorpresas 10
 - 8.2 Estrategia de pruebas 10
 - 8.3 Seguridad en cada fase 10
 - 8.4 Respaldos 10
 - 8.5 Definición de terminado (por fase) 10
- 9. Backlog de mejoras (después de la fase 5) 10

1. Cómo usar este plan

El sistema completo (51 requisitos funcionales) es un trabajo de varias fases. Este plan ordena ese trabajo de modo que **cada fase termina con algo que se puede desplegar y usar**, y cada paso tiene un criterio para saber que está terminado.

- **Tamaños relativos:** S (pocos días), M (una a dos semanas), L (varias semanas) para una persona. Son orientativos y dependen de lo que revele cada spike; no incluyo fechas.
- **Spike:** trabajo corto de investigación al inicio de una fase para confirmar lo que el motor realmente ofrece antes de construir encima.
- **Regla de oro:** no se avanza de fase hasta cumplir los criterios de aceptación y pasar el script de verificación del contrato.

1.1 Estado de partida

Elemento	Estado
Motor v0.6.9: binario oficial, checksum fijado, imagen y compose para Dokploy	Hecho
Mission Control: grafo vivo, actividad, aprobaciones, inspector, chat, presupuesto, integridad	Hecho
Login, proxy con lista blanca, scripts de verificación y de carga de agentes	Hecho
Pruebas contra el motor real (servidor, proxy, SSE)	Hecho
Build de Docker, despliegue en Dokploy y revisión en navegador	No probado

1.2 Mapa de fases

Fase	Resultado	Tamaño	Depende de
0. Poner en marcha	Studio y motor corriendo en tu VPS con HTTPS	S	—
1. Cerrar Mission Control	Fase 1 validada en navegador y móvil, con pruebas automáticas	S	0
2. Eventos, trazas y costos	Historial, línea de tiempo, detalle fino, costos, pausa	L	1
3. Constructor y asistente	Crear y editar agentes con clics y con IA	L	2
4. Operación	Hands, programación, canales, MCP, alertas, PWA	L	2
5. Confianza y escala	Auditoría, capacidades, evals, roles, replay, exportación	L	3 y 4

2. Fase 0: poner en marcha en tu VPS (Dokploy)

Objetivo: Studio y OpenFang funcionando con HTTPS y un primer conjunto de agentes. **Tamaño:** S.

Pasos

1. Sube el proyecto a un repositorio Git privado (git init, commit, push).
2. Genera los secretos: dos valores con openssl rand -hex 32 (OPENFANG_API_KEY y STUDIO_SECRET) y elige una STUDIO_PASSWORD larga.
3. En Dokploy crea un servicio de tipo Compose, conecta el repositorio y fija la ruta ./deploy/docker-compose.dokploy.yml.
4. En Environment pega las variables de .env.example con tus valores y al menos una clave de proveedor.
5. Si no usas Anthropic, edita deploy/openfang/config.toml (provider, model, api_key_env) antes de desplegar.
6. Crea el registro DNS A del subdominio hacia la IP del VPS.
7. En Domains agrega el subdominio con Service Name studio, puerto 3000 y HTTPS. No agregues dominio ni puertos al servicio openfang.

8. Despliega y revisa los logs: el build descarga el binario y verifica el SHA-256; si falla, versión y hash no coinciden.
9. Ejecuta dentro del contenedor del motor `scripts/verify_contract.py` (debe terminar en TODO OK) y `scripts/seed_agents.py` para crear agentes de ejemplo.
10. Entra a Studio, inicia sesión y comprueba que los agentes aparecen y que el motor figura como conectado.

Criterios de aceptación

- Studio responde por HTTPS y pide contraseña; sin sesión, cualquier ruta devuelve redirección o 401.
- El puerto 4200 del motor no es accesible desde fuera del VPS.
- `verify_contract.py` termina en TODO OK dentro del contenedor.
- Los agentes de ejemplo aparecen y el chat de prueba responde (con clave de proveedor válida).

Pruebas

- Desde tu equipo: curl al dominio sin cookie debe devolver 401 o 307; curl a IP:4200 debe fallar.
- Desde el VPS: `docker ps` muestra `openfang healthy` y `studio healthy`.

Riesgos y decisiones

- El build de Docker y el despliegue en Dokploy no se probaron: puede haber ajustes menores (nombre de la red `dokploy-network`, rutas relativas del `compose`). La guía `docs/DOKPLOY.md` trae una tabla de problemas comunes.
- Guarda las claves en Dokploy, no en el repositorio (`.env` está en `.gitignore`).

3. Fase 1: cerrar Mission Control

Objetivo: Validar lo construido en un navegador real y blindarlo con pruebas automáticas. **Tamaño:** S.

Pasos

1. Abre Studio en escritorio y móvil y recorre este guion: login, grafo, selección de nodo, inspector, chat, aprobaciones, pestañas, cambio entre claro y oscuro.
2. Anota defectos visuales o de interacción y corrígelos (zoom y encuadre del grafo, textos largos, estados vacíos).
3. Fuerza cada estado para verlo: detén un agente (detenido), envía un mensaje con proveedor válido (pensando), provoca una aprobación (ámbar), apaga el motor (sin conexión).
4. Añade pruebas unitarias del normalizador de estado (reglas de la sección 5.4 de la ERS) con datos simulados.
5. Añade pruebas de contrato en CI: descarga el binario v0.6.9, verifica el checksum, arranca el motor, ejecuta `verify_contract.py` y una prueba del proxy con lista blanca.
6. Añade una prueba end-to-end con Playwright: login, aparición de un agente, aprobación de una acción.
7. Define la configuración de CI (GitHub Actions) con build, pruebas y análisis de tipos.

Criterios de aceptación

- El guion completo pasa en Chrome y Safari móvil.
- Los seis estados de nodo se ven y se distinguen sin depender solo del color (texto y forma).
- CI en verde en cada cambio.

Pruebas

- Pruebas unitarias, de contrato (contra el binario real) y una end-to-end.
- Revisión de accesibilidad: teclado, foco, contraste, reducción de movimiento.

Riesgos y decisiones

- La posición de los nodos sigue en el navegador (RF-10 llega en la fase 2).

4. Fase 2: eventos, trazas y costos

Objetivo: Ver el historial de lo que hizo cada agente, con detalle fino, costos y control de pausa. **Tamaño:** L.

Pasos

1. Spike de eventos: abre el WebSocket `/api/agents/{id}/ws?token=...` de un agente, envía un mensaje y registra todos los tipos de evento (`typing`, `phase`, `tool_start`, `tool_end`, `text_delta`, `response`). Escribe `docs/eventos.md` con los hallazgos. Recuerda el límite de 5 conexiones por IP.
2. Define el formato normalizado de eventos (`agent_state`, `llm`, `tool_call`, `tool_result`, `approval_required`, `message`, `error`, `budget`) en `lib/events.ts`.
3. Añade Postgres al compose y un sistema de migraciones SQL (carpeta `migrations`, ejecutadas al arrancar).
4. Crea un colector que corra dentro del servidor de Studio aunque no haya navegadores abiertos (hook de instrumentación de Next.js): sondea la auditoría y las aprobaciones y guarda eventos normalizados. Escribe pruebas con datos simulados.
5. Agrega un WebSocket único hacia el agente seleccionado en el inspector y úsalo para marcar herramienta iniciada y terminada con precisión; si no basta, documenta el hueco para el spike del motor.
6. Crea las rutas `/api/runs` y `/api/runs/{id}` y la pantalla Ejecuciones con línea de tiempo por carriles (un carril por agente, bloques por paso, costo y tokens).
7. Amplía el inspector: resumen en lenguaje natural (primero con plantilla determinista a partir de la configuración), memoria con búsqueda, límites con barras de uso y conexiones con estado.
8. Construye la pantalla Costos con los endpoints de presupuesto y uso (por agente, modelo y día), proyección del mes y alerta visual al acercarse al límite.

9. Implementa pausa por agente y pausa global con confirmación, usando stop y start; registra quién la ejecutó (se completa en la fase 5).
10. Mueve posiciones y vistas guardadas del navegador a Postgres; agrega agrupación, colapso y filtros al grafo.
11. Muestra burbujas con los últimos mensajes entre agentes al pasar el cursor sobre una arista.

Criterios de aceptación

- Toda ejecución queda en el historial con sus pasos, costo y duración.
- Una herramienta iniciada se refleja en el nodo en menos de 3 segundos.
- La pausa global detiene todos los agentes y es reversible.
- El grafo con 30 agentes sigue siendo fluido y legible.

Pruebas

- Pruebas del normalizador y del colector con secuencias de eventos reales grabadas.
- Prueba de contrato de los endpoints nuevos.
- Prueba de carga: 30 agentes simulados.

Riesgos y decisiones

- Si el motor no emite los eventos que se necesitan, la solución es un endpoint pequeño en el motor (contribuible al proyecto); se decide en el spike.
- Las aristas entre pares dependen del historial de 500 eventos del motor: al persistir eventos en Postgres se elimina esa limitación.

5. Fase 3: constructor visual y asistente

Objetivo: Crear y editar agentes arrastrando y con ayuda de IA, sin tocar archivos. **Tamaño:** L.

Pasos

1. Spike de edición: probar qué permiten los endpoints PATCH `/api/agents/{id}/config`, `/update`, `/model`, `/tools`, `/skills`, `/mcp_servers` e `/identity` y si conviene editar en el lugar o recrear el agente desde un manifiesto. Documenta la decisión.
2. Modela el manifiesto en TypeScript con validación (esquema zod) y convierte a y desde TOML; cubre con pruebas usando los manifiestos oficiales de ejemplo.
3. Agrega al canvas una paleta (agentes, subagentes, herramientas, skills, MCP, canales, puertas de aprobación) con arrastrar y soltar; las conexiones inválidas se rechazan con una explicación.
4. Crea el panel de propiedades con formularios por tipo de nodo (modelo y parámetros, instrucciones, permisos, límites, herramientas).
5. Implementa el flujo Crear agente: plantilla o en blanco, formulario guiado, vista previa del grafo, creación con POST `/api/agents`.
6. Construye el componente de diff visual (nodos nuevos en verde, modificados en ámbar, eliminados en rojo) y el comparador de versiones lado a lado; guarda versiones de manifiestos en Postgres.
7. Añade el modo avanzado: editor del manifiesto TOML en paralelo, sincronizado con el diagrama y con errores por línea.
8. Asistente de creación: un agente dedicado dentro de OpenFang (por ejemplo, studio-architect) cuyo prompt produce un manifiesto TOML; Studio lo valida, lo muestra como diff y nunca lo aplica sin tu confirmación.
9. Editor de skills (SKILL.md) y de Hands (HAND.toml) con plantillas y prueba aislada; importación desde un repositorio.
10. Catálogo de plantillas con vista previa del grafo antes de instalar.

Criterios de aceptación

- Se puede crear un agente con subagentes solo con clics y que arranque.
- El asistente produce un manifiesto válido a partir de una descripción y muestra el diff.
- Ningún cambio se aplica sin una confirmación explícita.
- Cada cambio queda versionado y se puede comparar.

Pruebas

- Pruebas de ida y vuelta TOML a modelo a TOML con todos los manifiestos oficiales.
- Pruebas del validador con manifiestos rotos.
- Prueba end-to-end: crear agente con asistente, aceptar diff y ver el nodo.

Riesgos y decisiones

- La calidad del asistente depende del modelo que uses y de su costo: define un presupuesto para el agente asistente.
- Un manifiesto generado por IA puede pedir permisos amplios: el diff debe resaltar los permisos y capacidades nuevas.

6. Fase 4: operación (Hands, programación, canales, alertas, móvil)

Objetivo: Operar el sistema día a día desde la interfaz, incluso desde el teléfono. **Tamaño:** L.

Pasos

1. Pantalla Hands: lista, estado, activar y pausar instancias, métricas y su cola de aprobaciones (endpoints `/api/hands`).

2. Calendario de tareas programadas (cron y schedules): último resultado, próxima ejecución y ejecución manual.
3. Editor visual de workflows con historial de ejecuciones (/api/workflows).
4. Pantalla Canales: estado, configuración por formulario, enrutamiento a agentes y QR de WhatsApp (/api/channels y los endpoints de QR).
5. Catálogo de integraciones y servidores MCP con conexión guiada y estado de salud.
6. Motor de alertas dentro del colector: reglas por bucle, error repetido, agente caído y gasto cercano al límite; entrega por correo, Telegram o notificación push.
7. Edición de límites de presupuesto por agente desde la interfaz.
8. PWA: manifiesto, service worker, instalación en el móvil y notificaciones Web Push (claves VAPID) para aprobaciones y alertas.
9. Vista móvil optimizada de la bandeja de aprobaciones con contexto ampliado (por qué lo pide, argumentos, historial reciente).

Criterios de aceptación

- Se aprueba desde el teléfono una acción de un Hand y el agente continúa.
- Una alerta de gasto llega en menos de un minuto tras superar el umbral.
- WhatsApp se conecta escaneando el QR desde Studio.

Pruebas

- Pruebas de reglas de alertas con series simuladas.
- Prueba manual en iOS y Android de la instalación y las notificaciones.

Riesgos y decisiones

- Las notificaciones push requieren HTTPS y, en iOS, que la PWA esté instalada.
- Cada integración nueva amplía la superficie de ataque: revisar permisos antes de activarla.

7. Fase 5: confianza y escala

Objetivo: Auditar, controlar accesos, probar versiones y reproducir ejecuciones. **Tamaño:** L.

Pasos

1. Visor de la cadena de auditoría con filtros por agente, acción y resultado, y verificación de integridad bajo demanda.
2. Mapa de capacidades por agente con semáforo, a partir de su configuración y del estado de seguridad del motor.
3. Registro de auditoría propio de Studio: quién aprobó, detuvo, creó o cambió qué, con fecha y origen.
4. Usuarios y roles (propietario, administrador, editor, lector) con una librería de autenticación y Postgres; protege cada ruta del BFF según el rol.
5. Evals: casos con entrada y expectativas (contiene texto, usó cierta herramienta) que se ejecutan contra un agente antes de activar una versión, con semáforo de resultados.
6. Replay: reproducir una ejecución guardada animando el grafo con los eventos persistidos.
7. Bifurcación: desde un paso, clonar el agente (endpoint de clonado) y repetir con otro modelo o instrucciones; comparar resultados lado a lado.
8. Comentarios sobre nodos y ejecuciones.
9. Exportar e importar un agente como paquete; migración guiada desde otros frameworks con la herramienta del motor.
10. Documentación de operación: respaldos, actualización del motor, rotación de claves y recuperación ante desastres.

Criterios de aceptación

- Se audita una ejecución de punta a punta y se verifica la integridad de la cadena.
- Un usuario lector no puede aprobar ni detener nada.
- Dos versiones de un agente se comparan con evals y se ve cuál mejora.
- Una ejecución se reproduce sobre el grafo.

Pruebas

- Pruebas de autorización por rol en cada ruta.
- Pruebas de evals con casos conocidos.
- Prueba de restauración de respaldos.

Riesgos y decisiones

- OpenFang es un motor de un solo inquilino: los espacios de trabajo de Studio separan vistas y permisos, pero no aíslan datos. Para aislar clientes de verdad, usa un motor por cliente (un compose por cliente).
- Bifurcar clona un agente: cuida el costo de repetir ejecuciones.

8. Prácticas transversales

8.1 Actualizar el motor sin sorpresas

1. Lee las notas de la versión nueva y busca cambios incompatibles.
2. Prueba en un entorno aparte: construye la imagen con OPENFANG_VERSION y OPENFANG_SHA256 nuevos.
3. Ejecuta scripts/verify_contract.py y la suite de contrato de CI.
4. Si hay diferencias, adapta solo el BFF (lib/of.ts, app/api/events y el proxy).
5. Despliega en producción y deja la versión anterior disponible para volver atrás.

8.2 Estrategia de pruebas

Nivel	Qué cubre	Cuándo
Unitarias	Normalizador de eventos, reglas de estado, conversor TOML, reglas de alertas	En cada cambio
Contrato	Respuestas reales del motor fijado (campos y rutas que usa Studio)	En CI y antes de actualizar el motor
Integración	BFF contra el binario real: login, proxy, SSE	En CI
End-to-end	Flujos de usuario con Playwright	Antes de cada fase
Manual	Móvil, accesibilidad, notificaciones	Cierre de fase

8.3 Seguridad en cada fase

- El motor nunca se publica: sus GET públicos exponen agentes, aprobaciones, configuración y auditoría.
- Cada ruta nueva del proxy se agrega a la lista blanca con una prueba de que las demás siguen en 403.
- Ninguna clave llega al navegador; las respuestas se revisan para evitar fugas.
- Las acciones destructivas piden confirmación y quedan en la auditoría propia (fase 5).
- Rotar OPENFANG_API_KEY y STUDIO_SECRET periódicamente y tras cualquier sospecha.

8.4 Respaldos

- Volumen /data del motor (agentes, memoria, auditoría): copia diaria.
- Postgres de Studio (desde la fase 2): copia diaria y prueba de restauración mensual.

8.5 Definición de terminado (por fase)

1. Criterios de aceptación cumplidos y demostrados.
2. Pruebas nuevas en CI y en verde.
3. verify_contract.py sin fallos contra el motor fijado.
4. Documentación actualizada (README, DOKPLOY, ERS con el estado de cada requisito).
5. Desplegado en el VPS y revisado en móvil.

9. Backlog de mejoras (después de la fase 5)

- Resumen de agentes generado por IA (reemplaza la plantilla determinista de la fase 2).
- Aprendizaje de reglas de alertas a partir del historial.
- Modo presentación para demostraciones a clientes.
- Plantillas por industria con skills y Hands preconfigurados.
- Aplicación de escritorio con la misma interfaz.
- Contribuir al proyecto OpenFang los endpoints de eventos que se hayan añadido.